

PANDAS FOR BEGINNERS TO INTERMEDIATE

Module 1: Creating DataFrames and Reading CSV Files

What is Pandas?

Pandas is a Python library used for:

- Reading CSV/Excel files
- Cleaning data
- Filtering data
- Analyzing data
- Joining datasets
- Creating reports

Think of Pandas as **Excel inside Python**.

What is a DataFrame?

A DataFrame is a table consisting of rows and columns.

Example:

StudentID Name Marks

S1	Ravi	85
S2	Meera	90
S3	Karan	78

This table is called a **DataFrame** in Pandas.

Import Pandas

Before using Pandas:

```
import pandas as pd
```

Why pd? Because typing: `pandas.read_csv()` every time is long. Instead: `pd.read_csv()`

Function 1: `pd.DataFrame()`

Purpose

Creates a DataFrame manually.

Syntax

```
pd.DataFrame(data)
```

Example

```
import pandas as pd
```

```
data = {  
    "Name":["Ravi","Meera","Karan"],  
    "Marks":[85,90,78]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

Output

	Name	Marks
0	Ravi	85
1	Meera	90
2	Karan	78

Step-by-Step Explanation

Step 1

```
data = {  
    "Name":["Ravi","Meera","Karan"],  
    "Marks":[85,90,78]  
}
```

Creates a Python dictionary.

Memory:

```
{  
    "Name":["Ravi","Meera","Karan"],  
    "Marks":[85,90,78]  
}
```

Step 2

```
pd.DataFrame(data)
```

Converts dictionary → table.

Why Use It?

Useful when data is already available inside Python.

Common Mistake

Wrong:

```
data = {  
    "Name":["Ravi","Meera"],  
    "Marks":[85,90,78]  
}
```

Output:

ValueError

Reason:

Columns must contain equal number of values.

Practice Question 1

Create:

Product	Price
---------	-------

Pen	10
-----	----

Notebook	50
----------	----

Pencil	5
--------	---

Solution

```
data = {  
    "Product":["Pen","Notebook","Pencil"],  
    "Price":[10,50,5]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

Function 2: `pd.read_csv()`

Purpose

Reads CSV file into DataFrame.

Sample File

books.csv

BookID,Title,Price

B101,Python Basics,499

B102,Data Science,799

B103,SQL Mastery,599

Syntax

```
pd.read_csv("filename.csv")
```

Example

```
import pandas as pd
```

```
books = pd.read_csv("books.csv")
```

```
print(books)
```

Output

BookID	Title	Price
B101	Python Basics	499
B102	Data Science	799
B103	SQL Mastery	599

What Happens Internally?

Step 1

Pandas opens file.

books.csv

Step 2

Reads rows.

B101

B102

B103

Step 3

Creates DataFrame.

books

Why Use It?

95% of real-world projects start with:

```
pd.read_csv()
```

Practice Question 2

Read:
products.csv
into:
products_df

Solution

```
products_df = pd.read_csv(  
    "products.csv"  
)
```

Function 3: head()

Purpose

Shows first 5 rows.

Syntax

df.head()

Example

books.head()

Output

First 5 rows

Why Use It?

Quick preview.
Instead of printing:
100000 rows
You see:
5 rows

Custom Rows

books.head(3)
Output:
First 3 rows

Practice Question 3

Display first 2 rows.

Solution

```
books.head(2)
```

Function 4: tail()

Purpose

Shows last rows.

Syntax

df.tail()

Example

`books.tail()`

Output

Last 5 rows

Custom

`books.tail(2)`

Shows last 2 rows.

Why Use It?

Useful when:

CSV contains 50000 rows

and you want to inspect ending records.

Practice Question 4

Display last 3 rows.

Solution

`books.tail(3)`

Function 5: shape

Purpose

Returns:

(rows, columns)

Syntax

`df.shape`

Notice:

No brackets.

Wrong:

`df.shape()`

Correct:

`df.shape`

Example

`books.shape`

Output:

(100, 5)

Meaning:

100 rows

5 columns

Why Use It?

To know dataset size.

Practice Question 5

Find total rows and columns.

Solution

`books.shape`

Function 6: columns

Purpose

Returns all column names.

Syntax

`df.columns`

Example

`books.columns`

Output:

```
Index([
  'BookID',
  'Title',
  'Price'
])
```

Why Use It?

Useful when dataset has:

50 columns

and you need exact names.

Practice Question 6

Display all column names.

Solution

`books.columns`

Function 7: info()

Purpose

Shows:

- Column names
 - Data types
 - Missing values
 - Memory usage
-

Syntax

`df.info()`

Example

`books.info()`

Output:

RangeIndex: 100 entries

BookID object

Title object

Price int64

Understanding Data Types

object

Text

Ravi

Mumbai

Python

int64

Integers

10

20

30

float64

Decimals

4.5

8.2

Why Use It?

Before analysis always run:

`df.info()`

because it quickly tells:

- Missing values
- Wrong datatypes

Practice Question 7

Display complete dataset information.

Solution

`books.info()`

Function 8: `describe()`

Purpose

Statistical summary.

Works on numeric columns.

Syntax

`df.describe()`

Example

```
books.describe()
```

Output

```
count
mean
std
min
25%
50%
75%
max
```

Explanation

Suppose:
Price

```
100
200
300
400
500
```

count

Total records
5

mean

Average
 $(100+200+300+400+500)/5$

```
300
```

min

```
100
```

max

```
500
```

Why Use It?

Quick statistical understanding.

Practice Question 8

Display statistical summary of:

```
Price
Stock
Rating
```

Solution

books.describe()

Practice Dataset

Create:

books.csv

BookID,Title,Category,Price,Stock,Rating

B101,Python Basics,Programming,499,25,4.5

B102,Data Science 101,Data Science,799,15,4.8

B103,SQL Mastery,Database,599,10,4.2

B104,Machine Learning,Data Science,999,8,4.9

B105,Web Development,Programming,699,20,4.3

Assignment 1

1. Read CSV file
 2. Display first 3 rows
 3. Display last 2 rows
 4. Find shape
 5. Display columns
 6. Display info()
 7. Display describe()
 8. Create same DataFrame manually using pd.DataFrame()
-

PANDAS FOR BEGINNERS TO INTERMEDIATE

MODULE 2 : COLUMN SELECTION, ROW SELECTION, FILTERING & QUERYING

Dataset Used

books.csv

BookID,Title,Author,Category,Price,Stock,Rating

B101,Python Basics,John Smith,Programming,499,25,4.5

B102,Data Science 101,Alice Brown,Data Science,799,15,4.8

B103,SQL Mastery,John Smith,Database,599,10,4.2

B104,Machine Learning,David Lee,Data Science,999,8,4.9

B105,Web Development,Sarah Khan,Programming,699,20,4.3

Load Dataset

import pandas as pd

books = pd.read_csv("books.csv")

PART 1 : SELECTING COLUMNS

Function 1 : Single Column Selection

Syntax

df["column_name"]

Example

```
books["Title"]
```

Output

```
0    Python Basics
1    Data Science 101
2      SQL Mastery
3    Machine Learning
4    Web Development
```

What Happens Internally?

DataFrame:

Title

Python Basics

Data Science 101

SQL Mastery

Pandas extracts only Title column.

Why Use It?

Suppose dataset contains 50 columns.

You only need:

Title

No need to view everything.

Practice Question

Display:

Author

column only.

Solution

```
books["Author"]
```

Function 2 : Multiple Column Selection

Syntax

```
df[
    ["col1","col2"]
]
```

Notice:

```
[[ ]]
```

Double brackets.

Example

```
books[
    ["Title","Price"]
]
```

Output

Title	Price
Python Basics	499

Title	Price
Data Science 101	799

Why Double Brackets?

Single bracket:

```
books["Title"]
```

Returns Series.

Double bracket:

```
books[["Title"]]
```

Returns DataFrame.

Practice Question

Display:

Title

Rating

Solution

```
books[
    ["Title", "Rating"]
]
```

PART 2 : ROW SELECTION USING `iloc[]`

What is `iloc`?

`iloc` means:

Integer Location

Uses row numbers.

First Row

```
books.iloc[0]
```

Output

```
BookID    B101
```

```
Title     Python Basics
```

```
...
```

Second Row

```
books.iloc[1]
```

Third Row

```
books.iloc[2]
```

Why?

Rows start from:

0

1

2

3

4

not

1

2

3

4

5

Practice Question

Display 4th row.

Solution

`books.iloc[3]`

Multiple Rows

Syntax

`df.iloc[start:end]`

Example

`books.iloc[0:3]`

Output

Rows:

0

1

2

Important Rule

Ending index excluded.

Like:

`range(0,3)`

Practice Question

Display the first 4 rows.

Solution

`books.iloc[0:4]`

Row and Column Together

Syntax

`df.iloc[row,column]`

Example

`books.iloc[1,2]`

Output

Alice Brown

Explanation:

Row 1

Data Science 101
Column 2
Author

Practice Question

Display value at:
Row 3
Column 4

Solution

books.iloc[3,4]
Output
999

PART 3 : loc[]

What is loc?

loc means:
Location
Uses labels.

Example

```
books.loc[  
    :,  
    ["Title","Price"]  
]
```

Output

Title	Price
Python Basics	499

Understanding :

:
means
All Rows

["Title","Price"]
means
Only these columns

Example

```
books.loc[  
    0:2,  
    ["Title","Price"]  
]
```

Output

Rows:
0
1

2

Columns:

Title

Price

Practice Question

Display:

Rows 1-3

Columns:

Title

Rating

Solution

```
books.loc[
    1:3,
    ["Title", "Rating"]
]
```

PART 4 : FILTERING DATA

Most Important Topic in Pandas.

Filter 1 : Greater Than

Example

Books costing more than ₹700.

```
books[
    books["Price"] > 700
]
```

Output

Title	Price
Data Science 101	799
Machine Learning	999

Step by Step

Pandas checks:

499 > 700

False

799 > 700

True

999 > 700

True

Only True rows remain.

Practice Question

Find books with:

Rating > 4.5

Solution

```
books[
    books["Rating"] > 4.5
]
```

Filter 2 : Less Than

Example

```
books[
    books["Stock"] < 15
]
```

Output

SQL Mastery

Machine Learning

Practice Question

Find books with:

Price < 600

Solution

```
books[
    books["Price"] < 600
]
```

Filter 3 : Equal To

Example

```
books[
    books["Category"]=="Programming"
]
```

Output

Python Basics

Web Development

Practice Question

Display all:

Data Science

books.

Solution

```
books[
    books["Category"]=="Data Science"
]
```

PART 5 : MULTIPLE CONDITIONS

AND Condition (&)

Example

Price > 500

AND

Rating > 4.5

```
books[
  (books["Price"] > 500)
  &
  (books["Rating"] > 4.5)
]
```

Output

Data Science 101

Machine Learning

Why Parentheses?

Wrong

```
books[
  books["Price"] > 500 &
  books["Rating"] > 4.5
]
```

Error.

Always use:

```
(
)
&
(
)
```

Practice Question

Find books:

Price > 600

AND

Stock > 10

Solution

```
books[
  (books["Price"] > 600)
  &
  (books["Stock"] > 10)
]
```

OR Condition (|)

Example

```
books[
  (books["Category"]=="Programming")
  |
]
```

```
(books["Category"]=="Database")
]
```

Output

Programming books

Database books

Practice Question

Display books:

Price > 900

OR

Rating > 4.7

Solution

```
books[
    (books["Price"] > 900)
    |
    (books["Rating"] > 4.7)
]
```

PART 6 : isin()

Purpose

Check multiple values.

Example

Without isin()

```
books[
    (books["Category"]=="Programming")
    |
    (books["Category"]=="Database")
]
```

Better:

```
books[
    books["Category"].isin(
        ["Programming","Database"]
    )
]
```

Output

Programming Books

Database Books

Why Use It?

Cleaner syntax.

Practice Question

Show books from:

Programming

Data Science

Solution

```
books[
  books["Category"].isin(
    ["Programming","Data Science"]
  )
]
```

PART 7 : between()

Purpose

Range filtering.

Example

Books priced between:

500 and 800

```
books[
  books["Price"].between(
    500,
    800
  )
]
```

Output

799

599

699

Practice Question

Find ratings between:

4.2 and 4.8

Solution

```
books[
  books["Rating"].between(
    4.2,
    4.8
  )
]
```

PART 8 : query()

Purpose

Alternative filtering syntax.
Many Data Analysts prefer it.

Example

```
books.query(  
    "Price > 700"  
)
```

Equivalent to:

```
books[  
    books["Price"] > 700  
]
```

Multiple Conditions

```
books.query(  
    "Price > 500 and Rating > 4.5"  
)
```

Why Use It?

Cleaner.
Easier to read.

Practice Question

Find books:
Stock > 10

and

Rating > 4.3

Solution

```
books.query(  
    "Stock > 10 and Rating > 4.3"  
)
```

Assignment 2

Using books.csv:

Q1

Display only Title.

Q2

Display Title and Price.

Q3

Display first 3 rows using `iloc`.

Q4

Display rows 1–4 using `loc`.

Q5

Find books with Price > 700.

Q6

Find books with Rating > 4.5.

Q7

Find Programming books.

Q8

Find books with:

Price > 600

AND

Stock > 10

Q9

Find books with:

Programming

OR

Database

Q10

Use isin() for:

Programming

Data Science

Q11

Find books priced between:

500 and 800

Q12

Use query() to find:

Rating > 4.5

PANDAS FOR BEGINNERS TO INTERMEDIATE**MODULE 3 : DATA CLEANING (MOST IMPORTANT MODULE)**

Why Data Cleaning?

In real-world projects, data is rarely perfect.

You may find:

- ✗ Missing values
- ✗ Duplicate rows
- ✗ Wrong data types
- ✗ Extra spaces
- ✗ Incorrect values

Before analysis, we must clean data.

A common saying:

80% of a Data Analyst's time is spent cleaning data and 20% analyzing it.

Sample Dataset**employees.csv**

EmpID,Name,Department,Salary,Experience

E101,Ravi,IT,50000,3

E102,Meera,HR,,5

E103,Karan,IT,45000,

E104,Sneha,Finance,60000,4

E104,Sneha,Finance,60000,4

E105,,Marketing,55000,2

Load Dataset

import pandas as pd

```
employees = pd.read_csv(
    "employees.csv"
)
```

Function 1 : isnull()

Purpose

Find missing values.

Syntax

df.isnull()

Example

employees.isnull()

Output

EmpID Name Department Salary Experience

False False False False False

False False False True False

False False False False True

False False False False False

False False False False False

False True False False False

Understanding Output

False

Value exists.

True

Value missing.

Real Example

Salary

for Meera is missing.

Output:

True

Why Use It?

To identify problematic columns.

Practice Question

Find all missing values.

Solution

employees.isnull()

Function 2 : isnull().sum()

Purpose

Count missing values.

Syntax

df.isnull().sum()

Example

employees.isnull().sum()

Output

EmpID 0

Name 1

Department 0

Salary 1

Experience 1

How Does It Work?

Step 1

isnull()

creates:

True

False

True

Step 2

sum()

counts:

True = 1

False = 0

Why Use It?

Quickly identify columns needing cleaning.

Practice Question

Count missing values in dataset.

Solution

employees.isnull().sum()

Function 3 : notnull()

Purpose

Opposite of:

isnull()

Syntax

`df.notnull()`

Example

`employees.notnull()`

Output

True = value exists

False = missing

Why Use It?

Useful when filtering valid data.

Example

```
employees[
    employees["Salary"].notnull()
]
```

Only employees having salary.

Practice Question

Show employees whose experience exists.

Solution

```
employees[
    employees["Experience"].notnull()
]
```

Function 4 : dropna()

Purpose

Remove missing values.

Syntax

`df.dropna()`

Example

`employees.dropna()`

Output

Only complete rows remain

Before

6 rows

After

3 rows

because rows with missing values are removed.

How Does It Work?

Checks every row.

If row contains:

NaN

Remove it.

Why Use It?

When missing records are useless.

Practice Question

Remove all rows with missing values.

Solution

```
employees_clean = employees.dropna()
```

Common Mistake

Forgetting assignment.

Wrong:

```
employees.dropna()
```

Correct:

```
employees = employees.dropna()
```

Function 5 : fillna()

Purpose

Replace missing values.

Syntax

```
df.fillna(value)
```

Example

Replace missing salary with:

0

```
employees["Salary"] = (  
    employees["Salary"]  
    .fillna(0)  
)
```

Output

Salary

50000

0

45000

60000

60000

55000

Replace Missing Name

```
employees["Name"] = (  
    employees["Name"]  
    .fillna("Unknown")  
)
```

Output

Unknown

instead of blank.

Why Use It?

Removing rows may lose important data.

Better:

fillna()

Practice Question

Replace missing experience with:

0

Solution

```
employees["Experience"] = (  
    employees["Experience"]  
    .fillna(0)  
)
```

Function 6 : replace()

Purpose

Replace existing values.

Syntax

```
df.replace(old,new)
```

Example

```
employees["Department"] = (  
    employees["Department"]  
    .replace(  
        "IT",  
        "Technology"  
    )  
)
```

Output

Technology

instead of

IT

Why Use It?

Company changes naming conventions.

Practice Question

Replace:

HR

with

Human Resources

Solution

```
employees["Department"] = (  
    employees["Department"]  
    .replace(  
        "HR",  
        "Human Resources"  
    )  
)
```

Function 7 : duplicated()

Purpose

Find duplicate rows.

Syntax

```
df.duplicated()
```

Example

```
employees.duplicated()
```

Output

False

False

False

False

True

False

Meaning:

Row 5 is duplicate.

Why Use It?

Duplicate records create wrong reports.

Practice Question

Find duplicate rows.

Solution

```
employees.duplicated()
```

Function 8 : drop_duplicates()

Purpose

Remove duplicate rows.

Syntax

```
df.drop_duplicates()
```

Example

```
employees.drop_duplicates()
```

Before

6 rows

After

5 rows

Why Use It?

Duplicate employee records should not exist.

Practice Question

Remove duplicate rows.

Solution

```
employees = (  
    employees  
    .drop_duplicates()  
)
```

Function 9 : astype()

Purpose

Convert datatype.

Syntax

```
df["column"].astype(type)
```

Example

Convert Experience to integer.

```
employees["Experience"] = (  
    employees["Experience"]  
    .astype(int)  
)
```

Why?

Sometimes numbers become text.

Example:

Salary

50000
45000
60000
may be stored as:
object
instead of:
int64

Practice Question

Convert Salary to integer.

Solution

```
employees["Salary"] = (  
    employees["Salary"]  
    .astype(int)  
)
```

Function 10 : rename()

Purpose

Rename columns.

Syntax

```
df.rename(  
    columns={  
        old:new  
    }  
)
```

Example

```
employees.rename(  
    columns={  
        "Salary":"MonthlySalary"  
    }  
)
```

Output

MonthlySalary

Why Use It?

Better column names.

Practice Question

Rename:
Experience
to
YearsExperience

Solution

```
employees.rename(  
    columns={  
        "Experience":  
        "YearsExperience"  
    }  
)
```

Function 11 : pd.to_numeric()

Purpose

Convert text numbers into numeric values.

Suppose:

Salary

50000

45000

ABC

60000

Example

```
employees["Salary"] = (  
    pd.to_numeric(  
        employees["Salary"],  
        errors="coerce"  
    )  
)
```

Output

50000

45000

NaN

60000

Why Use errors="coerce"?

Invalid values become:

NaN

instead of crashing.

Practice Question

Convert Salary column into numeric.

Solution

```
employees["Salary"] = (  
    pd.to_numeric(  
        employees["Salary"],
```

```
        errors="coerce"
    )
)
```

Complete Cleaning Workflow

```
import pandas as pd
```

```
employees = pd.read_csv(
    "employees.csv"
)
```

```
print(
    employees.isnull().sum()
)
```

```
employees["Salary"] = (
    pd.to_numeric(
        employees["Salary"],
        errors="coerce"
    )
)
```

```
employees["Salary"] = (
    employees["Salary"]
    .fillna(0)
)
```

```
employees["Name"] = (
    employees["Name"]
    .fillna("Unknown")
)
```

```
employees = (
    employees
    .drop_duplicates()
)
```

```
print(employees)
```

Assignment 3

Using employees.csv

Q1

Find missing values.

Q2

Count missing values.

Q3

Show employees with salary available.

Q4

Remove rows containing missing values.

Q5

Replace missing salary with 0.

Q6

Replace missing name with "Unknown".

Q7

Find duplicate rows.

Q8

Remove duplicates.

Q9

Rename Salary → MonthlySalary.

Q10

Convert Salary to numeric.

Interview Questions

Q1

Difference between:

isnull()

and

notnull()

Answer

isnull()

Finds missing values.

notnull()

Finds available values.

Q2

Difference between:

dropna()

and

fillna()

Answer

dropna()

Removes records.

fillna()

Replaces missing values.

Q3

Difference between:

duplicated()

and

drop_duplicates()

Answer

duplicated()

Finds duplicates.

drop_duplicates()

Removes duplicates.

PANDAS FOR BEGINNERS TO INTERMEDIATE

MODULE 4 : SORTING, AGGREGATION & STATISTICAL FUNCTIONS

Why This Module is Important?

After cleaning data, the next step is:

- Find highest sales
- Find lowest sales
- Calculate averages
- Count records
- Find top customers
- Generate reports

These tasks are done using:

```
sort_values()  
sum()  
mean()  
max()  
min()  
count()  
value_counts()  
unique()  
nunique()  
nlargest()  
nsmallest()
```

Dataset Used

ecommerce_sales.csv

```
OrderID,Customer,Category,Amount,Quantity,City  
O101,Ravi,Electronics,25000,1,Mumbai  
O102,Meera,Fashion,3500,2,Pune  
O103,Karan,Electronics,15000,1,Mumbai  
O104,Sneha,Books,1200,3,Nashik  
O105,Ravi,Fashion,5000,2,Mumbai  
O106,Amit,Books,1800,4,Pune  
O107,Meera,Electronics,45000,1,Pune
```

Load Dataset

```
import pandas as pd  
  
sales = pd.read_csv(  
    "ecommerce_sales.csv"  
)
```

PART 1 : sort_values()

Purpose

Sort rows based on column values.

Syntax

```
df.sort_values("column")
```

Example

Sort Amount ascending.

```
sales.sort_values(  
    "Amount"  
)
```

Output

```
1200  
1800  
3500  
5000  
15000  
25000  
45000
```

How It Works

Before:

```
25000  
3500  
15000  
1200
```

After:

```
1200  
3500  
15000  
25000
```

Descending Order

Syntax

```
df.sort_values(  
    "Amount",  
    ascending=False  
)
```

Example

```
sales.sort_values(  
    "Amount",  
    ascending=False  
)
```

Output

```
45000  
25000  
15000  
5000
```

3500
1800
1200

Why Use It?

Find:

- Top customers
 - Highest salary
 - Highest sales
-

Practice Question

Sort Quantity descending.

Solution

```
sales.sort_values(  
    "Quantity",  
    ascending=False  
)
```

Multiple Column Sorting

Syntax

```
df.sort_values(  
    ["col1","col2"]  
)
```

Example

```
sales.sort_values(  
    ["Category","Amount"]  
)
```

Meaning:

Sort by Category first.

Inside Category sort by Amount.

Practice Question

Sort by:

City

then

Amount

Solution

```
sales.sort_values(  
    ["City","Amount"]  
)
```

PART 2 : sort_index()

Purpose

Sort DataFrame by row index.

Example

```
sales.sort_index()
```

Usually used after:

```
sort_values()
```

or

```
drop()
```

to restore order.

Practice Question

Sort DataFrame using index.

Solution

```
sales.sort_index()
```

PART 3 : sum()

Purpose

Calculate total.

Example

Total Sales Amount.

```
sales["Amount"].sum()
```

Calculation

25000

3500

15000

1200

5000

1800

45000

Total

96500

Why Use It?

Revenue calculation.

Practice Question

Find total quantity sold.

Solution

```
sales["Quantity"].sum()
```

Output

14

PART 4 : mean()

Purpose

Average value.

Example

```
sales["Amount"].mean()
```

Formula

Total Amount

/

Total Orders

Output

13785.71

Why Use It?

Average sales.

Average salary.

Average marks.

Practice Question

Find average quantity sold.

Solution

```
sales["Quantity"].mean()
```

PART 5 : median()

Purpose

Middle value.

Example

```
sales["Amount"].median()
```

Sorted:

1200

1800

3500

5000

15000

25000

45000

Middle:

5000

Why Use It?

Less affected by outliers.

Example

100

200

300

500000

Mean:

125150

Wrong impression.

Median:

250

More realistic.

Practice Question

Find median quantity.

Solution

```
sales["Quantity"].median()
```

PART 6 : mode()

Purpose

Most frequent value.

Example

```
sales["Category"].mode()
```

Output

Electronics

because it appears most.

Why Use It?

Most common:

- Product
 - City
 - Category
-

Practice Question

Find most common city.

Solution

```
sales["City"].mode()
```

PART 7 : max()

Purpose

Highest value.

Example

```
sales["Amount"].max()
```

Output

45000

Practice Question

Find highest quantity.

Solution

```
sales["Quantity"].max()
```

Output

4

PART 8 : min()

Purpose

Lowest value.

Example

```
sales["Amount"].min()
```

Output

1200

Practice Question

Find minimum quantity.

Solution

```
sales["Quantity"].min()
```

Output

1

PART 9 : count()

Purpose

Count non-null values.

Example

```
sales["Amount"].count()
```

Output

7

Difference

len(df)

counts rows.

count()

counts non-missing values.

Practice Question

Count customers.

Solution

```
sales["Customer"].count()
```

PART 10 : unique()

Purpose

Show unique values.

Example

```
sales["Category"].unique()
```

Output

```
[  
  'Electronics',  
  'Fashion',  
  'Books'  
]
```

Why Use It?

Understand available categories.

Practice Question

Show unique cities.

Solution

```
sales["City"].unique()
```

PART 11 : nunique()

Purpose

Count unique values.

Example

```
sales["Category"].nunique()
```

Output

```
3
```

Categories:

Electronics

Fashion

Books

Practice Question

Count unique customers.

Solution

```
sales["Customer"].nunique()
```

PART 12 : value_counts()

Purpose

Frequency counting.

Example

```
sales["Category"].value_counts()
```

Output

Electronics 3

Fashion 2

Books 2

Why Use It?

Very common interview question.

Practice Question

Count city frequency.

Solution

```
sales["City"].value_counts()
```

PART 13 : nlargest()

Purpose

Top N rows.

Example

Top 3 highest sales.

```
sales.nlargest(  
    3,  
    "Amount"  
)
```

Output

45000

25000

15000

Why Use It?

Find top customers.

Practice Question

Top 2 highest quantities.

Solution

```
sales.nlargest(  
    2,  
    "Quantity"
```

)

PART 14 : nsmallest()

Purpose

Lowest N rows.

Example

```
sales.nsmallest(  
    2,  
    "Amount"  
)
```

Output

1200

1800

Practice Question

Bottom 3 sales.

Solution

```
sales.nsmallest(  
    3,  
    "Amount"  
)
```

REAL BUSINESS REPORT

Suppose manager asks:

Total Revenue

```
sales["Amount"].sum()
```

Average Order Value

```
sales["Amount"].mean()
```

Highest Order

```
sales["Amount"].max()
```

Lowest Order

```
sales["Amount"].min()
```

Most Popular Category

```
sales["Category"].mode()
```

Top 5 Orders

```
sales.nlargest(  
    5,  
    "Amount"  
)
```

Assignment 4

Q1

Sort Amount ascending.

Q2

Sort Amount descending.

Q3

Sort by City then Amount.

Q4

Find total revenue.

Q5

Find average revenue.

Q6

Find median revenue.

Q7

Find highest order value.

Q8

Find lowest order value.

Q9

Find most common category.

Q10

Find unique cities.

Q11

Count unique customers.

Q12

Count category frequency.

Q13

Show top 3 orders.

Q14

Show bottom 2 orders.

Q15

Find total quantity sold.

Interview Questions

Q1

Difference between:

mean()

and

median()

Answer

Mean = Average

Median = Middle Value

Median handles outliers better.

Q2

Difference between:

unique()

and

nunique()

Answer

unique()
returns values.
nunique()
returns count.

Q3

Difference between:

sort_values()
and
sort_index()

Answer

sort_values()
sorts using column values.
sort_index()
sorts using row numbers.

PANDAS FOR BEGINNERS TO INTERMEDIATE**MODULE 5 : GROUPBY(), AGGREGATION & DATA SUMMARIZATION**

Why GroupBy is Important?

Imagine your manager asks:

Total sales category-wise?

Electronics → ?
Fashion → ?
Books → ?

Average sales city-wise?

Mumbai → ?
Pune → ?
Delhi → ?

Highest sale in each category?

Electronics → ?
Fashion → ?
Books → ?

These reports are created using:

groupby()

This is one of the most important Pandas functions.

Dataset Used**ecommerce_sales.csv**

OrderID, Customer, Category, Amount, Quantity, City
O101, Ravi, Electronics, 25000, 1, Mumbai
O102, Meera, Fashion, 3500, 2, Pune
O103, Karan, Electronics, 15000, 1, Mumbai
O104, Sneha, Books, 1200, 3, Nashik

O105,Ravi,Fashion,5000,2,Mumbai
O106,Amit,Books,1800,4,Pune
O107,Meera,Electronics,45000,1,Pune

Load Dataset

import pandas as pd

```
sales = pd.read_csv(  
    "ecommerce_sales.csv"  
)
```

What Does GroupBy Mean?

Suppose:

Customer Amount

Ravi 1000

Ravi 2000

Meera 3000

Manager asks:

Total sales by customer

Instead of:

Ravi

1000

2000

Meera

3000

You want:

Ravi 3000

Meera 3000

This grouping process is called:

groupby()

Function 1 : groupby()

Basic Syntax

```
df.groupby(  
    "column"  
)
```

Example

```
sales.groupby(  
    "Category"  
)
```

Output

GroupBy Object

Important

groupby() alone doesn't show results.

You must perform an operation.

Examples:

sum()

mean()

count()

max()

min()

Function 2 : groupby().sum()

Total Sales By Category

```
sales.groupby(  
    "Category"  
)["Amount"].sum()
```

Output

Books	3000
Electronics	85000
Fashion	8500

How Does It Work?

Electronics:

25000

15000

45000

Total:

85000

Fashion:

3500

5000

Total:

8500

Books:

1200

1800

Total:

3000

Practice Question

Find total quantity sold category-wise.

Solution

```
sales.groupby(  
    "Category"  
)["Quantity"].sum()
```

Function 3 : `groupby().mean()`

Average Sales By Category

```
sales.groupby(  
    "Category"  
)["Amount"].mean()
```

Output

```
Books      1500  
Electronics 28333.33  
Fashion     4250
```

Why Use It?

Average order value.
Average salary.
Average marks.

Practice Question

Find average quantity category-wise.

Solution

```
sales.groupby(  
    "Category"  
)["Quantity"].mean()
```

Function 4 : `groupby().count()`

Count Orders By Category

```
sales.groupby(  
    "Category"  
)["OrderID"].count()
```

Output

```
Books      2  
Electronics 3  
Fashion     2
```

Why Use It?

Find transaction count.

Practice Question

Count customers city-wise.

Solution

```
sales.groupby(  
    "City"  
)["Customer"].count()
```

Function 5 : `groupby().max()`

Highest Sale By Category

```
sales.groupby(  
    "Category"  
)["Amount"].max()
```

Output

```
Books      1800  
Electronics 45000  
Fashion     5000
```

Practice Question

Find maximum quantity category-wise.

Solution

```
sales.groupby(  
    "Category"  
)["Quantity"].max()
```

Function 6 : `groupby().min()`

Lowest Sale By Category

```
sales.groupby(  
    "Category"  
)["Amount"].min()
```

Output

```
Books      1200  
Electronics 15000  
Fashion     3500
```

Practice Question

Find minimum quantity category-wise.

Solution

```
sales.groupby(  
    "Category"  
)["Quantity"].min()
```

Function 7 : `groupby()` Multiple Columns

Group By City and Category

```
sales.groupby(  
    ["City", "Category"]  
)["Amount"].sum()
```

Output

Mumbai Electronics 40000

Mumbai Fashion 5000

Pune Electronics 45000

Pune Fashion 3500

Nashik Books 1200

Why Use It?

Manager asks:

Category-wise sales

inside each city

Practice Question

Find quantity sold by:

City

Category

Solution

```
sales.groupby(  
    ["City","Category"]  
)["Quantity"].sum()
```

Function 8 : reset_index()

Problem

Output of groupby:

```
sales.groupby(  
    "Category"  
)["Amount"].sum()
```

looks like:

Category

Books 3000

Fashion 8500

Category becomes index.

Convert Back To DataFrame

```
sales.groupby(  
    "Category"  
)["Amount"].sum().reset_index()
```

Output

Category	Amount
----------	--------

Books	3000
-------	------

Electronics	85000
-------------	-------

Fashion	8500
---------	------

Why Use It?

Most reporting tools require normal DataFrames.

Practice Question

Generate city-wise sales and reset index.

Solution

```
sales.groupby(  
    "City"  
)["Amount"].sum().reset_index()
```

Function 9 : as_index=False

Instead of:

```
groupby()  
.reset_index()
```

Use:

```
sales.groupby(  
    "Category",  
    as_index=False  
)["Amount"].sum()
```

Output

Normal DataFrame directly.

Practice Question

Create city-wise quantity report.

Solution

```
sales.groupby(  
    "City",  
    as_index=False  
)["Quantity"].sum()
```

Function 10 : agg()

Purpose

Perform multiple calculations together.

Example

```
sales.groupby(  
    "Category"  
)["Amount"].agg(  
    [  
        "sum",  
        "mean",  
        "max",
```

```
        "min"  
    ]  
)
```

Output

Category	sum	mean	max	min
Books	3000	1500	1800	1200
Electronics	85000	28333	45000	15000
Fashion	8500	4250	5000	3500

Why Use It?

Create complete reports in one line.

Practice Question

Generate:
sum
mean
max
for Quantity.

Solution

```
sales.groupby(  
    "Category"  
)["Quantity"].agg(  
    [  
        "sum",  
        "mean",  
        "max"  
    ]  
)
```

Function 11 : Multiple Column Aggregation

Example

```
sales.groupby(  
    "Category"  
)agg(  
{  
    "Amount": [  
        "sum",  
        "mean"  
    ],  
    "Quantity": [  
        "sum",  
        "max"  
    ]  
})
```

)

Output
Amount
 sum
 mean

Quantity
 sum
 max
for each category.

Real Business Example

Manager asks:

Category Report

Need:

Total Revenue

Average Revenue

Total Quantity

Maximum Quantity

One query:

```
sales.groupby(  
    "Category"
```

```
).agg(  
{
```

```
{
```

```
  "Amount":[
```

```
    "sum",
```

```
    "mean"
```

```
],
```

```
  "Quantity":[
```

```
    "sum",
```

```
    "max"
```

```
  ]
```

```
}  
}
```

```
)
```

Function 12 : Group Filtering

Find categories with sales > 10000.

Step 1

```
report = sales.groupby(  
    "Category"  
)["Amount"].sum()
```

Output

Books 3000
Electronics 85000
Fashion 8500

Step 2

```
report[  
    report > 10000  
]
```

Output

Electronics
85000

Practice Question

Find cities with sales > 20000.

Solution

```
city_sales = sales.groupby(  
    "City"  
)["Amount"].sum()
```

```
city_sales[  
    city_sales > 20000  
]
```

Complete Business Dashboard

```
print(  
    "Total Revenue:"  
)
```

```
print(  
    sales["Amount"].sum()  
)
```

```
print(  
    "\nCategory Wise Revenue:"  
)
```

```
print(  
    sales.groupby(  
        "Category"  
    )["Amount"].sum()  
)
```

```
print(  
    "\nCity Wise Revenue:"  
)
```

```

print(
    sales.groupby(
        "City"
    )["Amount"].sum()
)

print(
    "\nCategory Summary:"
)

print(
    sales.groupby(
        "Category"
    ).agg(
        {
            "Amount":[
                "sum",
                "mean",
                "max"
            ],
            "Quantity":[
                "sum",
                "mean"
            ]
        }
    )
)

```

Assignment 5

Q1

Find total sales category-wise.

Q2

Find total quantity category-wise.

Q3

Find average sales category-wise.

Q4

Find maximum sale category-wise.

Q5

Find minimum sale category-wise.

Q6

Count orders category-wise.

Q7

Find city-wise sales.

Q8

Find city-wise quantity.

Q9

Find city-category sales report.

Q10

Use agg() to calculate:

sum
mean
max
min
for Amount.

Q11

Create report using:

as_index=False

Q12

Find categories with sales > 10000.

Interview Questions

Q1

Why doesn't groupby() show results directly?

Answer

Because it only creates groups.

Need:

sum()

mean()

count()

max()

to perform calculations.

Q2

Difference between:

value_counts()

and

groupby().count()

Answer

value_counts()

Counts frequencies of one column.

groupby().count()

Provides grouped reporting and works with multiple columns.

Q3

Why use reset_index()?

Answer

Convert grouped index back into normal DataFrame columns.

MODULE 5 - ASSIGNMENT SOLUTIONS

Using:

OrderID, Customer, Category, Amount, Quantity, City

O101, Ravi, Electronics, 25000, 1, Mumbai

O102, Meera, Fashion, 3500, 2, Pune

O103, Karan, Electronics, 15000, 1, Mumbai

O104, Sneha, Books, 1200, 3, Nashik

O105, Ravi, Fashion, 5000, 2, Mumbai

O106, Amit, Books, 1800, 4, Pune

O107,Meera,Electronics,45000,1,Pune

import pandas as pd

```
sales = pd.read_csv(
    "ecommerce_sales.csv"
)
```

Q1 Total Sales Category Wise

```
sales.groupby(
    "Category"
)["Amount"].sum()
```

Output

Books	3000
Electronics	85000
Fashion	8500

Q2 Total Quantity Category Wise

```
sales.groupby(
    "Category"
)["Quantity"].sum()
```

Output

Books	7
Electronics	3
Fashion	4

Q3 Average Sales Category Wise

```
sales.groupby(
    "Category"
)["Amount"].mean()
```

Output

Books	1500
Electronics	28333.33
Fashion	4250

Q4 Maximum Sale Category Wise

```
sales.groupby(
    "Category"
)["Amount"].max()
```

Output

Books	1800
Electronics	45000
Fashion	5000

Q5 Minimum Sale Category Wise

```
sales.groupby(
    "Category"
)["Amount"].min()
```

Output

Books	1200
Electronics	15000
Fashion	3500

Q6 Count Orders Category Wise

```
sales.groupby(  
    "Category"  
)["OrderID"].count()  
Output  
Books      2  
Electronics 3  
Fashion     2
```

Q7 City Wise Sales

```
sales.groupby(  
    "City"  
)["Amount"].sum()  
Output  
Mumbai 45000  
Nashik  1200  
Pune    50300
```

Q8 City Wise Quantity

```
sales.groupby(  
    "City"  
)["Quantity"].sum()  
Output  
Mumbai 4  
Nashik 3  
Pune    7
```

Q9 City + Category Report

```
sales.groupby(  
    ["City","Category"]  
)["Amount"].sum()  
Output  
Mumbai Electronics 40000  
Mumbai Fashion     5000  
  
Nashik Books      1200  
  
Pune Books        1800  
Pune Electronics  45000  
Pune Fashion      3500
```

Q10 Agg()

```
sales.groupby(  
    "Category"
```

```

)["Amount"].agg(
[
    "sum",
    "mean",
    "max",
    "min"
]
)

```

Output

Category	sum	mean	max	min
Books	3000	1500	1800	1200
Electronics	85000	28333.33	45000	15000
Fashion	8500	4250	5000	3500

Q11 as_index=False

```

sales.groupby(
    "Category",
    as_index=False
)["Amount"].sum()

```

Output

Category	Amount
Books	3000
Electronics	85000
Fashion	8500

Q12 Categories With Sales > 10000

```

report = sales.groupby(
    "Category"
)["Amount"].sum()

```

```

report[
    report > 10000
]

```

Output

Electronics	85000
-------------	-------

MODULE 6

MERGING, JOINING & CONCATENATING DATAFRAMES

This is one of the MOST IMPORTANT topics in Data Analytics.

Real companies never keep everything in one file.

Example:

customers.csv

CustomerID, CustomerName, City
C101, Ravi, Mumbai
C102, Meera, Pune
C103, Karan, Nashik
C104, Sneha, Delhi

orders.csv

OrderID, CustomerID, Amount
O101, C101, 5000
O102, C102, 3000
O103, C101, 7000
O104, C105, 2500

Why Merge?

Customers file:

CustomerID Name

C101 Ravi

Orders file:

OrderID CustomerID

O101 C101

Need:

OrderID CustomerName Amount

O101 Ravi 5000

This is called:

merge()

Function 1 : merge()

Syntax

```
pd.merge(  
    left_df,  
    right_df,  
    on="common_column"  
)
```

INNER JOIN

Most common join.

Only matching records.

Example

```
pd.merge(  
    customers,  
    orders,  
    on="CustomerID",  
    how="inner"  
)
```

Output

CustomerID	CustomerName	OrderID	Amount
------------	--------------	---------	--------

C101	Ravi	O101	5000
C101	Ravi	O103	7000
C102	Meera	O102	3000

Notice:

C105

removed.

Because customer not found.

LEFT JOIN

Keep ALL rows from left table.

```
pd.merge(
    customers,
    orders,
    on="CustomerID",
    how="left"
```

```
)
```

Output

CustomerID	CustomerName	OrderID
------------	--------------	---------

C101	Ravi	O101
C101	Ravi	O103
C102	Meera	O102
C103	Karan	NaN
C104	Sneha	NaN

Why NaN?

No orders.

RIGHT JOIN

Keep ALL rows from right table.

```
pd.merge(
    customers,
    orders,
    on="CustomerID",
    how="right"
```

```
)
```

Output

Includes:

C105

with missing customer details.

OUTER JOIN

Keep everything.

```
pd.merge(
```

```
customers,  
orders,  
on="CustomerID",  
how="outer"  
)
```

Output

All customers

-

All orders

Function 2 : concat()

Purpose:

Stack DataFrames.

Example

january.csv

OrderID,Amount

O101,5000

O102,7000

february.csv

OrderID,Amount

O103,6000

O104,8000

```
pd.concat(  
    [jan,feb]  
)
```

Output

OrderID Amount

O101 5000

O102 7000

O103 6000

O104 8000

ignore_index

```
pd.concat(  
    [jan,feb],  
    ignore_index=True  
)
```

Resets row numbers.

Function 3 : join()

Suppose:

```
customers.set_index(  
    "CustomerID"
```

```
)  
orders.set_index(  
    "CustomerID"  
)
```

Then:

```
customers.join(  
    orders,  
    how="inner"  
)
```

Produces merged output.

Difference

merge()

Most common

Uses columns.

join()

Uses index.

concat()

Stacks DataFrames.

Practice Dataset

products.csv

ProductID,ProductName,Category

P101,Laptop,Electronics

P102,Mouse,Electronics

P103,Book,Education

sales.csv

SaleID,ProductID,Quantity

S101,P101,2

S102,P102,5

S103,P105,1

Assignment 6

Q1

Perform inner join.

Q2

Perform left join.

Q3

Perform right join.

Q4

Perform outer join.

Q5

Find products with no sales.

Q6

Find sales with invalid ProductID.

Q7

Combine January and February sales using concat().

Q8

Use join() after setting ProductID as index.

Q9

Count total sales quantity product-wise.

Q10

Find product names along with quantity sold.

Solutions**Q1 Inner Join**

```
pd.merge(  
    products,  
    sales,  
    on="ProductID",  
    how="inner"  
)
```

Q2 Left Join

```
pd.merge(  
    products,  
    sales,  
    on="ProductID",  
    how="left"  
)
```

Q3 Right Join

```
pd.merge(  
    products,  
    sales,  
    on="ProductID",  
    how="right"  
)
```

Q4 Outer Join

```
pd.merge(  
    products,  
    sales,  
    on="ProductID",  
    how="outer"  
)
```

Q5 Products With No Sales

```
report = pd.merge(  
    products,  
    sales,
```

```
    on="ProductID",
    how="left"
)
```

```
report[
    report["SaleID"].isnull()
]
```

Output

P103 Book

Q6 Invalid ProductID

```
report = pd.merge(
    products,
    sales,
    on="ProductID",
    how="right"
)
```

```
report[
    report["ProductName"].isnull()
]
```

Output

P105

Q7 Concat

```
pd.concat(
    [jan_sales,feb_sales],
    ignore_index=True
)
```

Q8 Join

```
products = products.set_index(
    "ProductID"
)
```

```
sales = sales.set_index(
    "ProductID"
)
```

```
products.join(
    sales,
    how="inner"
)
```

Q9 Product Wise Quantity

```
sales.groupby(
    "ProductID"
)["Quantity"].sum()
```

Q10 Product Name + Quantity

```
merged = pd.merge(  
    products.reset_index(),  
    sales.reset_index(),  
    on="ProductID"  
)  
  
merged[  
    ["ProductName", "Quantity"]  
]
```

PANDAS FOR BEGINNERS TO INTERMEDIATE

MODULE 7 : PIVOT TABLES, CROSSTAB, APPLY(), MAP(), LAMBDA FUNCTIONS

Why This Module Is Important?

Until now we have learned:

- ✓ Filtering
- ✓ GroupBy
- ✓ Aggregation
- ✓ Merge

Now we will learn functions that are heavily used in:

- Dashboards
- Excel Reports
- Business Intelligence
- Data Analytics Interviews

Dataset Used

sales.csv

```
OrderID, Customer, Category, City, Amount, Quantity  
O101, Ravi, Electronics, Mumbai, 25000, 1  
O102, Meera, Fashion, Pune, 3500, 2  
O103, Karan, Electronics, Mumbai, 15000, 1  
O104, Sneha, Books, Nashik, 1200, 3  
O105, Ravi, Fashion, Mumbai, 5000, 2  
O106, Amit, Books, Pune, 1800, 4  
O107, Meera, Electronics, Pune, 45000, 1
```

PART 1 : pivot_table()

What is Pivot Table?

Excel users already know Pivot Table.

Suppose manager asks:

Revenue by City and Category

Instead of:

Mumbai Electronics 25000

Mumbai Electronics 15000

Mumbai Fashion 5000

Need:

City	Books	Electronics	Fashion
Mumbai	0	40000	5000
Pune	1800	45000	3500
Nashik	1200	0	0

This is Pivot Table.

Syntax

```
pd.pivot_table(  
    df,  
    values="Amount",  
    index="City",  
    columns="Category"  
)
```

Example

```
pivot = pd.pivot_table(  
    sales,  
    values="Amount",  
    index="City",  
    columns="Category",  
    aggfunc="sum"  
)
```

```
print(pivot)
```

Output

City	Books	Electronics	Fashion
Mumbai	NaN	40000	5000
Nashik	1200	NaN	NaN
Pune	1800	45000	3500

Why aggfunc="sum"?

Because multiple rows exist.

Pandas must know:

Sum?

Average?

Maximum?

Practice Question

Create quantity pivot.

Solution

```
pd.pivot_table(  
    sales,  
    values="Quantity",
```

```
index="City",
columns="Category",
aggfunc="sum"
)
```

Fill Missing Values

Problem

Pivot shows:
NaN

Solution

```
pd.pivot_table(
    sales,
    values="Amount",
    index="City",
    columns="Category",
    aggfunc="sum",
    fill_value=0
)
```

Output

City	Books	Electronics	Fashion
Mumbai	0	40000	5000
Nashik	1200	0	0
Pune	1800	45000	3500

PART 2 : Crosstab

What is Crosstab?

Counts combinations.

Example:

Manager asks:

Number of Orders

City vs Category

Syntax

```
pd.crosstab(
    df["City"],
    df["Category"]
)
```

Example

```
pd.crosstab(
    sales["City"],
    sales["Category"]
)
```

)

Output

City	Books	Electronics	Fashion
Mumbai	0	2	1
Nashik	1	0	0
Pune	1	1	1

Why Use It?

Count relationships.

Common HR Example:

Gender vs Department

Practice Question

Create Customer vs Category Crosstab.

Solution

```
pd.crosstab(  
    sales["Customer"],  
    sales["Category"]  
)
```

PART 3 : map()

Purpose

Replace values quickly.

Example

Suppose:

```
sales["City"]
```

contains:

Mumbai

Pune

Nashik

Need:

MH

MH

MH

Syntax

```
column.map(dictionary)
```

Example

```
city_codes = {  
    "Mumbai":"MH",  
    "Pune":"MH",  
    "Nashik":"MH"
```

```
}  
  
sales["State"] = (  
  sales["City"]  
  .map(city_codes)  
)
```

Output

City	State
Mumbai	MH
Pune	MH

Practice Question

Convert category names to codes.

```
{  
  "Electronics":"E",  
  "Fashion":"F",  
  "Books":"B"  
}
```

Solution

```
codes = {  
  "Electronics":"E",  
  "Fashion":"F",  
  "Books":"B"  
}
```

```
sales["CategoryCode"] = (  
  sales["Category"]  
  .map(codes)  
)
```

PART 4 : apply()

Purpose

Apply function on a column.

Example

Create GST Amount.
GST = 18%

Without apply:

```
sales["GST"] = (  
  sales["Amount"] * 0.18  
)
```

Using apply:

```
sales["GST"] = (  
    sales["Amount"]  
    .apply(  
        lambda x: x * 0.18  
    )  
)
```

Understanding Lambda

lambda x: x * 0.18

Means:

For every value x
multiply by 0.18

Example

25000

becomes

4500

Practice Question

Create Discount Column.

Rules:

Amount > 10000

10% Discount

Otherwise

5% Discount

Solution

```
sales["Discount"] = (  
    sales["Amount"]  
    .apply(  
        lambda x:  
            x*0.10  
            if x>10000  
            else x*0.05  
    )  
)
```

PART 5 : apply() With Functions

Instead of lambda:

def classify(amount):

if amount > 20000:
 return "High"

```
elif amount > 5000:  
    return "Medium"
```

```
return "Low"
```

```
sales["SalesType"] = (  
    sales["Amount"]  
    .apply(classify)  
)
```

Output

Amount SalesType

25000 High

3500 Low

15000 Medium

Why Use This?

More readable.

Used in real projects.

PART 6 : apply() Row Wise

Example

Need:

TotalValue =

Amount × Quantity

```
sales["TotalValue"] = (  
    sales.apply(  
        lambda row:  
            row["Amount"] *  
            row["Quantity"],  
        axis=1  
    )  
)
```

Why axis=1?

axis=0 → column wise

axis=1 → row wise

Example

For first row:

25000 × 1

Output

25000

Practice Question

Create:

AveragePrice

Amount / Quantity

Solution

```
sales["AveragePrice"] = (  
    sales.apply(  
        lambda row:  
            row["Amount"] /  
            row["Quantity"],  
        axis=1  
    )  
)
```

PART 7 : value_counts() vs crosstab()

value_counts()

One column.

```
sales["Category"].value_counts()
```

Output

Electronics 3

Fashion 2

Books 2

Crosstab

Two columns.

```
pd.crosstab(  
    sales["City"],  
    sales["Category"]  
)
```

Output

City vs Category

PART 8 : Ranking

Top Sales Ranking

```
sales["Rank"] = (  
    sales["Amount"]  
    .rank(  
        ascending=False  
    )  
)
```

Output

Amount Rank

45000	1
25000	2
15000	3

Practice Question

Rank quantity.

Solution

```
sales["QuantityRank"] = (  
    sales["Quantity"]  
    .rank(  
        ascending=False  
    )  
)
```

MINI PROJECT

Manager wants dashboard.

Category Wise Revenue

```
sales.groupby(  
    "Category"  
)["Amount"].sum()
```

City Wise Revenue

```
sales.groupby(  
    "City"  
)["Amount"].sum()
```

Revenue Pivot

```
pd.pivot_table(  
    sales,  
    values="Amount",  
    index="City",  
    columns="Category",  
    aggfunc="sum",  
    fill_value=0  
)
```

Order Count

```
pd.crosstab(  
    sales["City"],  
    sales["Category"]  
)
```

GST Calculation

```
sales["GST"] = (  

```

```
sales["Amount"]
    .apply(
        lambda x:x*0.18
    )
)
```

Rank Orders

```
sales["Rank"] = (
    sales["Amount"]
    .rank(
        ascending=False
    )
)
```

Assignment 7

Q1

Create city-category revenue pivot.

Q2

Create city-category quantity pivot.

Q3

Create city-category crosstab.

Q4

Map city names to state code.

Q5

Map category names to short codes.

Q6

Create GST column (18%).

Q7

Create discount column.

Q8

Create SalesType column.

Q9

Create TotalValue column.

Q10

Create AveragePrice column.

Q11

Rank orders by Amount.

Q12

Rank orders by Quantity.

Assignment 7 Solutions

Q1

```
pd.pivot_table(
    sales,
    values="Amount",
    index="City",
    columns="Category",
    aggfunc="sum",
```

```
    fill_value=0  
)
```

Q2

```
pd.pivot_table(  
    sales,  
    values="Quantity",  
    index="City",  
    columns="Category",  
    aggfunc="sum",  
    fill_value=0  
)
```

Q3

```
pd.crosstab(  
    sales["City"],  
    sales["Category"]  
)
```

Q4

```
sales["State"] = (  
    sales["City"]  
    .map(  
        {  
            "Mumbai":"MH",  
            "Pune":"MH",  
            "Nashik":"MH"  
        })  
)
```

Q5

```
sales["Code"] = (  
    sales["Category"]  
    .map(  
        {  
            "Electronics":"E",  
            "Fashion":"F",  
            "Books":"B"  
        })  
)
```

Q6

```
sales["GST"] = (  
    sales["Amount"]  
    .apply(  
        lambda x:x*0.18  
    )  
)
```

Q7

```
sales["Discount"] = (  
    sales["Amount"]  
    .apply(  
        lambda x:  
            x*0.10  
            if x>10000  
            else x*0.05  
    )  
)
```

Q8

```
def classify(amount):  
  
    if amount > 20000:  
        return "High"  
  
    elif amount > 5000:  
        return "Medium"  
  
    return "Low"  
  
sales["SalesType"] = (  
    sales["Amount"]  
    .apply(classify)  
)
```

Q9

```
sales["TotalValue"] = (  
    sales.apply(  
        lambda row:  
            row["Amount"] *  
            row["Quantity"],  
        axis=1  
    )  
)
```

Q10

```
sales["AveragePrice"] = (  
    sales.apply(  
        lambda row:  
            row["Amount"] /  
            row["Quantity"],  
        axis=1  
    )  
)
```

Q11

```
sales["AmountRank"] = (
    sales["Amount"]
    .rank(
        ascending=False
    )
)
```

Q12

```
sales["QuantityRank"] = (
    sales["Quantity"]
    .rank(
        ascending=False
    )
)
```

PANDAS FOR BEGINNERS TO INTERMEDIATE**MODULE 8 : ADVANCED PANDAS & INTERVIEW PREPARATION****What You'll Learn**

In this module, you'll learn advanced Pandas functions that are frequently used in:

- Data Analyst Interviews
 - Data Cleaning Projects
 - Dashboard Development
 - Business Reporting
 - ETL Pipelines
 - Data Science Projects
-

Dataset 1 : Monthly Sales Report

Month,Laptop,Mobile,Tablet

Jan,100,200,50

Feb,120,220,60

Mar,150,250,75

Load:

```
import pandas as pd
```

```
sales = pd.read_csv(
    "monthly_sales.csv"
)
```

PART 1 : melt()**Problem**

Current data:

Month Laptop Mobile Tablet

Jan 100 200 50

Feb 120 220 60

This is called:

Wide Format

Many dashboards need:

Long Format

Syntax

```
pd.melt(  
    df,  
    id_vars=[],  
    value_vars=[]  
)
```

Example

```
pd.melt(  
    sales,  
    id_vars="Month",  
    value_vars=[  
        "Laptop",  
        "Mobile",  
        "Tablet"  
    ]  
)
```

Output

Month variable value

Jan	Laptop	100
Jan	Mobile	200
Jan	Tablet	50
Feb	Laptop	120

Why Use It?

Machine Learning and Visualization tools often need long format data.

Practice Question

Convert product sales into long format.

Solution

```
pd.melt(  
    sales,  
    id_vars="Month"  
)
```

PART 2 : pivot()

Purpose

Opposite of melt()

Convert:

Long
to
Wide

Example Data

Month Product Sales

Jan	Laptop	100
Jan	Mobile	200
Feb	Laptop	120

Syntax

```
df.pivot(  
    index,  
    columns,  
    values  
)
```

Example

```
df.pivot(  
    index="Month",  
    columns="Product",  
    values="Sales"  
)
```

Output

Month Laptop Mobile

Jan	100	200
Feb	120	220

Difference

pivot()

No duplicate combinations allowed.

pivot_table()

Handles duplicates.

Interview Question

When should you use:

`pivot_table()`

instead of

`pivot()`

Answer:

When duplicate records exist.

PART 3 : `stack()`

Purpose

Convert columns into rows.

Example

Before

Month Laptop Mobile

Jan	100	200
-----	-----	-----

```
sales.set_index(  
    "Month"  
)
```

Output

Month

Jan Laptop 100

Jan Mobile 200

Why Use It?

Advanced reshaping.

Rare but common in interviews.

PART 4 : unstack()

Opposite of stack.

```
sales.set_index(  
    "Month"  
)
```

Returns original table.

PART 5 : explode()

Dataset

```
data = {  
    "Customer":[  
        "Ravi",  
        "Meera"  
    ],  
  
    "Products":[  
        ["Laptop","Mouse"],  
        ["Book","Pen"]  
    ]  
}
```

```
df = pd.DataFrame(data)
```

Output

Customer Products

Ravi [Laptop,Mouse]

Meera [Book,Pen]

Problem

Need:

Customer Products

Ravi Laptop

Ravi Mouse

Meera Book

Meera Pen

Solution

```
df.explode(  
    "Products"  
)
```

Why Use It?

E-commerce orders.

Tags.

Multiple selections.

Survey analysis.

Practice Question

Explode skills list.

Solution

```
df.explode(  
    "Skills"  
)
```

PART 6 : Datetime Functions

Sample Dataset

OrderID,OrderDate

O101,2025-01-15

O102,2025-02-10

O103,2025-03-20

Convert To Datetime

```
orders["OrderDate"] = (  
    pd.to_datetime(  
        orders["OrderDate"]  
    )  
)
```

Why?

Without conversion:

object

After conversion:

datetime64

Extract Year

```
orders["Year"] = (  
    orders["OrderDate"]  
    .dt.year  
)
```

Output

2025

Extract Month

```
orders["Month"] = (  
    orders["OrderDate"]  
    .dt.month  
)
```

Output

1

2

3

Month Name

```
orders["MonthName"] = (  
    orders["OrderDate"]  
    .dt.month_name()  
)
```

Output

January

February

March

Day Name

```
orders["Day"] = (  
    orders["OrderDate"]  
    .dt.day_name()  
)
```

Output

Wednesday

Monday

Practice Questions

Extract Year

Extract Month

Extract Day Name

PART 7 : String Functions

Dataset

```
employees["Name"]
```

Output

ravi

MEERA

Karan

Upper Case

```
employees["Name"].str.upper()
```

Output

RAVI

MEERA

KARAN

Lower Case

```
employees["Name"].str.lower()
```

Title Case

```
employees["Name"].str.title()
```

Output

Ravi

Meera

Karan

contains()

Find names containing:

"a"

```
employees[  
    employees["Name"]  
    .str.contains("a")  
]
```

Output

Ravi

Karan

startswith()

```
employees[  
    employees["Name"]  
    .str.startswith("R")  
]
```

endswith()

```
employees[  
    employees["Name"]  
    .str.endswith("i")  
]
```

]

split()

```
employees["FullName"].str.split()
```

Example

Ravi Sharma

becomes

```
["Ravi","Sharma"]
```

replace()

```
employees["Email"] = (
```

```
    employees["Email"]
```

```
    .str.replace(
```

```
        "@gmail.com",
```

```
        ""
```

```
    )
```

```
)
```

Practice Questions

Convert names to uppercase.

Find names containing "a".

Find names starting with "M".

Remove ".com" from emails.

PART 8 : Window Functions

Very common in analytics.

Dataset

sales

Amount

100

200

300

400

cumsum()

Running total.

```
sales["Amount"].cumsum()
```

Output

100

300

600

1000

Why Use It?

Revenue tracking.

cummax()

Running maximum.

```
sales["Amount"].cummax()
```

Output

100

200

300

400

cummin()

Running minimum.

```
sales["Amount"].cummin()
```

cumcount()

```
sales.groupby(
```

```
    "Customer"
```

```
).cumcount()
```

Useful for:

Customer Visit Number

rolling()

Moving average.

Example

```
sales["Amount"].rolling(
```

```
    3
```

```
).mean()
```

Meaning:

Average of last 3 rows.

Example

100

200

300

Average

200

Why Use It?

Stock market analysis.

Trend analysis.

Forecasting.

PART 9 : Interview Questions

Q1

Difference between:

`pivot()`

and

`pivot_table()`

Answer:

`pivot()`

requires unique combinations.

`pivot_table()`

handles duplicates.

Q2

Difference between:

`map()`

and

`apply()`

Answer:

`map()`

works on Series.

`apply()`

works on Series and DataFrames.

Q3

Difference between:

`stack()`

and

`unstack()`

Answer:

`stack()`

columns → rows.

`unstack()`

rows → columns.

Q4

Why use:

`pd.to_datetime()`

Answer:

Converts text dates into datetime objects.

Enables:

`.dt.year`

`.dt.month`

`.dt.day`

Assignment 8

Q1

Convert sales table using melt().

Q2

Convert long format back using pivot().

Q3

Use stack().

Q4

Use unstack().

Q5

Explode product lists.

Q6

Extract year from OrderDate.

Q7

Extract month name.

Q8

Convert names to uppercase.

Q9

Find names containing "a".

Q10

Calculate cumulative sales.

Q11

Calculate rolling average.

Q12

Create monthly sales summary using pivot_table().

Assignment 8 Solutions

Q1

```
pd.melt(  
    sales,  
    id_vars="Month"  
)
```

Q2

```
long_df.pivot(  
    index="Month",  
    columns="Product",  
    values="Sales"  
)
```

Q3

```
sales.set_index(  
    "Month"  
)
```

Q4

```
sales.set_index(  
    "Month"  
)
```

Q5

```
df.explode(  
    "Products"  
)
```

Q6

```
orders["Year"] = (  
    orders["OrderDate"]  
    .dt.year  
)
```

Q7

```
orders["MonthName"] = (  
    orders["OrderDate"]  
    .dt.month_name()  
)
```

Q8

```
employees["Name"].str.upper()
```

Q9

```
employees[  
    employees["Name"]  
    .str.contains("a")  
]
```

Q10

```
sales["Amount"].cumsum()
```

Q11

```
sales["Amount"].rolling(  
    3  
) .mean()
```

Q12

```
pd.pivot_table(  
    sales,  
    values="Amount",  
    index="City",  
    columns="Category",  
    aggfunc="sum",  
    fill_value=0  
)
```

Final Pandas Roadmap Completed

You now know:

Data Loading

- read_csv()

- read_excel()
- DataFrame()

Inspection

- head()
- tail()
- info()
- describe()

Selection

- loc[]
- iloc[]

Filtering

- query()
- isin()
- between()

Cleaning

- isnull()
- dropna()
- fillna()
- astype()

Aggregation

- sum()
- mean()
- count()
- value_counts()

Grouping

- groupby()
- agg()

Joining

- merge()
- join()
- concat()

Reporting

- pivot_table()
- crosstab()

Transformation

- map()
- apply()
- lambda

Advanced

- melt()
- pivot()
- stack()
- unstack()
- explode()

Date & String Processing

- to_datetime()
- .dt
- .str

Analytics

- `cumsum()`
- `rolling()`
- `rank()`

This covers roughly **90–95% of Pandas used in beginner-to-intermediate Data Analyst, Python Developer, and entry-level Data Science interviews and projects.**